

机器学习基础与训练范式

机器学习是 A I 的核心分支，其核心目标是让计算机从数据中自动学习规律，并用于预测或决策。

机器学习基础与训练范式

机器学习是 A I 的核心分支，其核心目标是让计算机从数据中自动学习规律，并用于预测或决策。

- - -

一、机器学习的形式化定义

1 . 1 T o m M i t c h e l l 的经典定义

" 一个计算机程序从经验 E 中学习，针对某类任务 T，用性能度量 P 来衡量。如果在任务 T 上，经过经验 E 后，有所提高，则称该程序对任务 T 进行了学习。 "

示例：垃圾邮件分类

- 任务 T：将邮件分类为 " 垃圾 " 或 " 正常 "
- 经验 E：历史邮件及其标签
- 性能 P：分类准确率

1 . 2 机器学习的三大假设

- 1 . 数据可表示：现实世界的问题可以用数值特征表示
- 2 . 模式存在：数据中存在可学习的统计规律
- 3 . 未来可预测：训练数据的分布与未来数据的分布相似（I I D 假设）

当第三个假设被违反时（分布偏移），模型性能会急剧下降。这是 A I 落地中最常见的失败原因之一。

- - -

二、三大范式对比

2 . 1 直观类比

监督学习 → 有标准答案的考试

- 每个样本都有正确标签
- 模型通过大量带标样本训练

- 目标：最小化预测与标签之间的差异

无监督学习 → 自主整理仓库

- 所有样本都没有标签
- 系统自行发现数据的内在结构
- 目标：发现数据中的模式（聚类、降维）

强化学习 → 机器人自主送货

- 机器人通过与环境交互获得奖励
- 必须通过试错学习最优策略
- 目标：最大化长期累积奖励

2.2 数学形式化对比

维度 监督学习 无监督学习 强化学习

输入 特征 X + 标签 y 仅特征 X 状态 s

输出 预测 $\hat{y}$ 结构 / 模式 动作 a

反馈信号 真实标签（密集、即时） 无外部反馈 奖励 r （稀疏、延迟）

目标函数 最小化 $L(\hat{y}, y)$ 最大化内部一致性 最大化 $E[\sum \gamma^t r]$

训练方式 批量训练 批量训练 在线交互式训练

核心挑战 标注成本高 评估困难 奖励稀疏 / 信用分配

- - -

三、监督学习详解

3.1 经验风险最小化 vs 结构风险最小化

经验风险最小化（ERM）：

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i)$$

直接最小化训练集上的平均损失。问题是可能过拟合。

结构风险最小化（SRM）：

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i) + \lambda J(\theta)$$

在经验风险基础上加入正则化项 $\Omega(\theta)$ ，控制模型复杂度，防止过拟合。

3.2 偏差 - 方差权衡（详细推导）

对于平方损失，模型的期望泛化误差可以分解为：

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

其中：

- Bias^2 ：模型预测均值与真实值的偏差的平方 $= \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$
- Variance ：模型预测的方差 $= \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$
- σ^2 ：数据的不可约噪声

直觉理解：

偏差高（打偏了）：模型系统性地偏离真实值

→ 解决：增加模型复杂度、添加特征

方差高（打散了）：模型对训练数据的小变化过于敏感

→ 解决：增加训练数据、正则化、简化模型

理想：低偏差 + 低方差（又准又稳）

状态 偏差 方差 现象 解决方案

欠拟合 高 低 训练集和测试集表现都差 更复杂模型、更多特征

过拟合 低 高 训练集好但测试集差 正则化、更多数据、早停

理想 低 低 两边都好 恰当的模型复杂度

3.3 线性回归详解

模型： $\hat{y} = w^T x + b$

损失函数（MSE）： $L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

闭式解（最小二乘法）：

$$\hat{w} = (X^T X)^{-1} X^T y$$

为什么MSE是最小二乘法的自然选择？

从概率视角看，假设 $y = w^T x + b + \epsilon$ ，其中噪声 $\epsilon \sim \mathcal{N}(0, \sigma^2)$ ，则最大似然估计等价于最小化MSE。因此MSE不是随意选择的，而是在高斯噪声假设下自然导出的。

线性回归的四大假设（Gauss-Markov条件）：

假设 含义 违反后果 检验方法

线性性 Y与X是线性关系 模型偏差大 残差v.s拟合值图

独立性 误差项相互独立 标准误估计有偏 Durbin-Watson检验

同方差性 误差方差恒定 置信区间不准 Breusch-Pagan检验

正态性 误差服从正态分布 小样本推断不准 Q-Q图、Shapiro检验

正则化线性回归：

当特征之间存在多重共线性或特征过多时，闭式解 $(X^T X)^{-1}$ 可能不稳定或不存在。正则化通过约束参数来解决：

Ridge回归（L2正则化）：

$$\min_w \frac{1}{n} \sum_{i=1}^n (y_i - w^T x_i)^2 + \lambda \|w\|_2^2$$

闭式解变为： $\hat{w} = (X^T X + \lambda I)^{-1} X^T y$ ，其中 I 是单位矩阵，保证了矩阵始终可逆。

- 效果：将参数往0方向压缩，但不会精确等于0
- 适用：特征多且都有用，但系数值偏大时
- 直觉：像弹簧把所有系数往0拉，但拉力有限

Lasso回归（L1正则化）：

$$\min_w \frac{1}{n} \sum_{i=1}^n (y_i - w^T x_i)^2 + \lambda \|w\|_1$$

- 效果：将部分参数压缩到精确的0，实现特征选择
- 适用：特征多但只有少数真正有用时
- 直觉：像一把刀，直接砍掉不重要的特征

为什么 L1 产生稀疏解而 L2 不会？

L2 正则化的约束区域是圆： L1 正则化的约束区域是菱形：

/ \ / \
/ \ / \
/ ○ \ ← 最优解在边上 / ○ \ ← 最优解常在角上
/ \ / \
\ / \ /
\ / \ /
\ / \ /
\ / \ /

菱形的角在坐标轴上，所以最优解常出现在w某个分量 = 0 的位置 → 稀疏

ElasticNet (L1 + L2 混合)：

$$\min_w \frac{1}{n} \sum_{i=1}^n (y_i - w^T x_i)^2 +$$

结合两者优点：既能特征选择，又能处理相关特征组。

线性回归的多种求解方式

```
import numpy as np
from sklearn.linear_model import LinearRegression,
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
```

方式1：正规方程（闭式解）

```
X = np.array([[1], [2], [3], [4]])
y = np.array([2.1, 3.9, 6.2, 7.8])
```

添加偏置列

```
Xb = np.c_[np.ones((4, 1)), X]
wnormal = np.linalg.inv(Xb.T @ Xb) @ Xb.T @ y
print(f"正规方程解： w={wnormal[1]:.3f}, b={wnormal[0]:.3f}")
```

方式2：梯度下降

```
w = np.random.randn(2, 1)
lr = 0.01
for epoch in range(1000):
    ypred = Xb @ w
    gradient = 2/4 Xb.T @ (ypred - y.reshape(-1, 1))
    w -= lr * gradient

print(f"梯度下降解: w={w[1, 0]:.3f}, b={w[0, 0]:.3f}")
```

方式3：sklearn（推荐）

普通线性回归

```
lrmodel = LinearRegression().fit(X, y)
```

Ridge回归（L2正则化）

alpha越大，正则化越强，系数越接近0

```
ridge = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
ridge.fit(X.reshape(-1, 1), y)
```

Lasso回归（L1正则化）—— 特征选择

```
lasso = make_pipeline(StandardScaler(), Lasso(alpha=1.0))
lasso.fit(X.reshape(-1, 1), y)
```

ElasticNet（L1+L2）

```
enet = make_pipeline(StandardScaler(), ElasticNet(alpha=1.0))
enet.fit(X.reshape(-1, 1), y)
```

选择最佳正则化参数

```
from sklearn.model_selection import GridSearchCV
paramgrid = {'ridgealpha': [0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(make_pipeline(StandardScaler(), Ridge(alpha=1.0)),
                    paramgrid)
grid.fit(X.reshape(-1, 1), y)
print(f"最佳alpha: {grid.best_params}")
```

模型诊断——残差分析：

残差分析：检查模型假设是否成立

```
import matplotlib.pyplot as plt
```

```
ypred = model.predict(Xtest)
```

```
residuals = ytest - ypred
```

1. 残差 vs 拟合值图：检查线性性和同方差性

如果呈随机散布 → 假设成立

如果有曲线趋势 → 需要添加非线性项

如果漏斗形状 → 存在异方差

2. 残差 Q - Q 图：检查正态性

点近似在直线上 → 正态性成立

3. 残差 vs 各特征：检查是否有遗漏的非线性关系

3.4 逻辑回归详解

模型： $P(y=1|x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-w^T x - b}}$

损失函数（交叉熵）：

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

为什么不用 MSE 做分类？

1. MSE + Sigmoid → 梯度消失（当预测正确时梯度接近 0）

2. 交叉熵 + Sigmoid → 梯度始终与误差成正比，学习更高效

3. 交叉熵是 MLE 在伯努利分布假设下的自然结果

决策边界的概率解释：

决策边界 $w^T x + b = 0$

将特征空间分为两个区域。距离决策边界越远的点，预测概率越接近 0 或 1；边界附近的点概率接近 0.5，表示不确定性。

决策边界的几何含义： w 是决策边面的法向量， $\|w\|$

越大，概率从 0.5 到 1 的变化越陡峭（相当于 Sigmoid 变得更像阶跃函数）。

多分类逻辑回归（Softmax 回归）：

$$P(y=k|x) = \frac{e^{w_k^T x + b_k}}{\sum_{j=1}^K e^{w_j^T x + b_j}}$$

损失函数为多类交叉熵：

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log p_{ik}$$

逻辑回归的正则化：

`sklearn` 中 `LogisticRegression` 默认使用 L2 正则化，参数 `C` 是正则化强度

`C = 1.0` 是默认值

`C` 很大 (如 100) → 几乎不正则化 → 可能过拟合

`C` 很小 (如 0.01) → 强正则化 → 可能欠拟合

```
model = LogisticRegression(C=1.0, penalty='l2') # 默认
```

```
model = LogisticRegression(C=1.0, penalty='l1', solver='libsvm')
```

类不平衡处理：

当正负样本比例悬殊时，逻辑回归会偏向多数类。解决方案：

方法 原理 `sklearn` 参数

类权重调整 少数类的损失权重增大 `class_weight='balanced'`

过采样 复制少数类样本 `SMOTE`

欠采样 减少多数类样本 `RandomUnderSampler`

阈值调整 不用 0.5，选择最优阈值 PR 曲线找最优点

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import classification_report
```

处理类不平衡

```
model = LogisticRegression(class_weight='balanced')
```

```
model.fit(X_train, y_train)
```

查看权重：`class_weight = nsamples / (nclasses * np.bincount(y_train))`

如 1000 个样本，900 负 100 正 → 正类权重 = $1000 / (2 * 100) = 5$ ，负类权重 = 1

调整决策阈值

```

yprob = model.predict_proba(Xtest)[: , 1]
threshold = 0.3 # 降低阈值, 增加召回率
ypred = (yprob >= threshold).astype(int)

```

逻辑回归完整实现

```

class LogisticRegression:
    def __init__(self, lr=0.01, epochs=1000, lambda_reg=0.01):
        self.lr = lr
        self.epochs = epochs
        self.lambda_reg = lambda_reg # L2正则化系数

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

    def fit(self, X, y):
        nsamples, nfeatures = X.shape
        self.w = np.zeros(nfeatures)
        self.b = 0

        for i in range(self.epochs):
            z = X @ self.w + self.b
            ypred = self.sigmoid(z)

            # 梯度计算 (含L2正则化, 偏置不正则化)
            dw = (1/nsamples) * X.T @ (ypred - y) + self.lambda_reg * self.w
            db = (1/nsamples) * np.sum(ypred - y)

            # 参数更新
            self.w -= self.lr * dw
            self.b -= self.lr * db

    def predict_proba(self, X):
        return self.sigmoid(X @ self.w + self.b)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

```

3.5 SVM 详解

核心思想：找到使两类样本间隔最大的超平面。

最大间隔的几何直觉：

超平面 $w^T x + b = 0$ 将空间分为两类。间隔 (margin) 是离超平面最近的样本到超

$$\text{margin} = \frac{2}{\|w\|}$$

最大化间隔等价于最小化 $\|w\|$ ：

$$\max_{w, b} \frac{2}{\|w\|} \quad \text{s.t.}$$

等价于：

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s.t.}$$

为什么约束是 $y_i (w^T x_i + b) \geq 1$ 而不是 > 0 ？

通过缩放 w 和 b ，总可以使得最近的样本满足 $y_i (w^T x_i + b) = 1$ （这叫“函数间隔”归一化到1）。这样间隔就只由 $\|w\|$ 决定。

对偶问题：

利用拉格朗日乘子法，将约束优化转化为对偶问题：

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i, j}$$

$$\text{s.t.} \quad \alpha_i \geq 0, \quad \sum_i$$

对偶问题的关键性质：

- 目标函数只依赖内积 $x_i^T x_j$ ，这为核技巧奠定了基础
- KKT 互补松弛条件： $\alpha_i (y_i (w^T x_i + b) - 1) = 0$ 。只有支持向量 (> 0)，其余样本的 $\alpha_i = 0$
- 这意味着 SVM 的决策只依赖少数支持向量，具有稀疏性

软间隔 SVM（处理线性不可分）：

引入松弛变量 ξ_i ，允许部分样本违反间隔约束：

$$\min_{w, b, \{x_i\}} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - (w^T x_i + b))$$

C 是正则化参数： C 大 $\rightarrow$ 少容忍错误（可能过拟合）， C 小 $\rightarrow$ 多容忍错误（可能欠拟合）。

核技巧：当数据线性不可分时，将数据映射到高维空间。

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

关键：不需要显式计算 $\phi(x)$ ，只需计算核函数 K 。

核函数 公式 适用场景 直觉

线性核 $K(x, z) = x^T z$ 线性可分 原始空间分类

多项式核 $K(x, z) = (x^T z + c)^d$ 非线性特征组合 添加多项式特征

RBF核 $K(x, z) = \exp(-\gamma \|x - z\|^2)$ 通用非线性 相似度

Sigmoid核 $K(x, z) = \tanh(\alpha x^T z + c)$ 类似神经网络

RBF核的直觉： $K(x, z)$ 衡量 x 和 z 的相似度。当 x 和 z 很接近时 $K \rightarrow 1$ ， γ 控制衰减速度： γ 大 $\rightarrow$ 只有非常近的才算相似。

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV
```

SVM分类（推荐先标准化！）

```
clf = make_pipeline(StandardScaler(), SVC(kernel='rbf'))
clf.fit(X_train, y_train)
```

C 参数：正则化强度

C 大 $\rightarrow$ 更少容忍分类错误（可能过拟合）

C 小 $\rightarrow$ 更多容忍分类错误（可能欠拟合）

γ 参数：RBF核的影响范围

γ 大 $\rightarrow$ 每个样本影响范围小（可能过拟合）

γ 小 $\rightarrow$ 每个样本影响范围大（可能欠拟合）

超参数搜索

```
paramgrid = {  
    'svcc': [0.1, 1, 10, 100],  
    'svccgamma': ['scale', 0.01, 0.1, 1],  
}  
gridsearch = GridSearchCV(clf, paramgrid, cv=5, s  
gridsearch.fit(Xtrain, ytrain)  
print(f"最佳参数: {gridsearch.bestparams}")
```

3.6 决策树详解

核心思想：递归地选择最优特征进行分裂，将数据划分为越来越纯的子集。

信息熵的直觉：熵衡量数据的 " 混乱程度 "。纯数据（只有一类）熵为 0，均匀分布的数据熵最大。

$$H(D) = - \sum_{k=1}^K p_k \log_2 p_k$$

数据分布 熵值 含义

[1, 1, 1, 1] 0 完全纯净

[0.5, 0.5] 1.0 最混乱（二分类）

[0.9, 0.1] 0.47 较纯净

[0.25, 0.25, 0.25, 0.25] 2.0 最混乱（四分类）

分裂准则：

准则 公式 特点

信息增益 $IG = H(D) - \sum \frac{|D_v|}{|D|} H(D_v)$ ID3

信息增益率 $IGR = IG / IV$ C4.5 算法，克服多值偏好

基尼指数 $Gini(D) = 1 - \sum p_k^2$ CART 算法，计算更快

信息增益为什么偏好多值特征？

如果一个特征有很多值（如 " 日期 "），每个值对应的子集很小，子集可能很纯，信息增益很高，但这样的分裂有泛化能力。信息增益率通过除以 " 固有值 "（IV）来惩罚这种情况：

$$IV = - \sum_{v=1}^V \frac{|D_v|}{|D|} \log_2 \frac{|D_v|}{|D|}$$

信息熵和基尼指数的计算

```
import numpy as np

def entropy(y):
    """计算信息熵"""
    , counts = np.unique(y, returncounts=True)
    probs = counts / len(y)
    return -np.sum(probs * np.log2(probs + 1e-10))

def gini(y):
    """计算基尼指数"""
    , counts = np.unique(y, returncounts=True)
    probs = counts / len(y)
    return 1 - np.sum(probs ** 2)

def infogain(Xcol, y, threshold):
    """计算按threshold二分后的信息增益"""
    n = len(y)
    leftmask = Xcol <= threshold
    rightmask = ~leftmask

    if leftmask.sum() == 0 or rightmask.sum() == 0:
        return 0

    nleft, nright = leftmask.sum(), rightmask.sum()
    Hparent = entropy(y)
    Hleft = entropy(y[leftmask])
    Hright = entropy(y[rightmask])

    return Hparent - (nleft / n) * Hleft - (nright / n) * Hright
```

示例

```
y = [0, 0, 0, 1, 1, 1, 1, 1] # 3个类别0, 5个类别1
print(f"信息熵: {entropy(y):.4f}") # 0.9544
print(f"基尼指数: {gini(y):.4f}") # 0.4688
```

寻找最佳分裂点

```
Xcol = np.array([2.5, 3.0, 3.5, 4.0, 5.5, 6.0, 6.5])
thresholds = np.sort(np.unique((Xcol[:-1] + Xcol[1:])/2))
bestthreshold = max(thresholds, key=lambda t: infogain(Xcol, t))
print(f"最佳分裂点: {bestthreshold}, 信息增益: {infogain(Xcol, bestthreshold):.4f}")
```

决策树的剪枝：

决策树容易过拟合——树太深会“记住”训练数据的噪声。剪枝方法：

方法 原理 实现

预剪枝 分裂前判断是否应该分裂 maxdepth, minsamplessplit

后剪枝 先建完整棵树，再从底向上剪枝 cost-complexity pruning

正则化 限制树的复杂度 minsamplesleaf, maxleafnodes

代价复杂度剪枝（CCP）：

$$R(\alpha, T) = R(T) + \alpha |T|$$

其中 $R(T)$ 是训练误差， $|T|$ 是叶节点数， α 控制惩罚强度。

3.7 集成学习

集成学习是机器学习中最重要实践方法之一。核心思想：多个弱学习器组合成一个强学习器。

理论基础——Condorcet 陪审团定理：如果每个陪审员独立做出正确判断的概率大于 50%，则多数投票率随陪审员数量增加而趋向 100%。

三个关键条件：

1. 个体学习器要好于随机猜测（准确率 > 50%）
2. 学习器之间要有差异性（不同类型错误）
3. 组合方式要合理（投票、加权平均等）

3.7.1 Bagging (Bootstrap Aggregating)

训练多个独立模型，投票 / 平均。

Bootstrap 采样：从 n 个样本中有放回地随机抽取 n 个样本。约 63.2% 的原始样本会被采到 ($1 - 1/e \approx 0.632$)，未被采到的 36.8% 构成 OOB (Out-of-Bag)。

- 代表：随机森林
- 每棵树使用随机采样的数据和随机选择的特征
- OOB (Out-of-Bag) 估计：未被采样的数据用于评估，无需额外验证集
- 袋外误差是对泛化误差的无偏估计

随机森林的随机性来源：

1. 样本随机：每棵树使用不同的 Bootstrap 采样
2. 特征随机：每次分裂只考虑 $\sqrt{p}$ 个随机特征 (p 是总特征数)
3. 两种随机性的叠加使树之间的相关性大大降低

3.7.2 Boosting

序列训练多个模型，每个模型关注前一个模型的错误。

- 代表：AdaBoost、GBDT、XGBoost、LightGBM、CatBoost
- 核心思想：给之前分错的样本更高权重

AdaBoost 算法：

```
class AdaBoost:
    """AdaBoost 二分类器"""
    def __init__(self, nestimators=50):
        self.nestimators = nestimators
        self.alphas = [] # 每个弱学习器的权重
        self.models = [] # 弱学习器列表

    def fit(self, X, y):
        n = len(y)
        weights = np.ones(n) / n # 初始化样本权重

        for t in range(self.nestimators):
            # 训练弱学习器 (决策树桩)
            stump = DecisionStump()
```

```

stump.fit(X, y, sampleweight=weights)
ypred = stump.predict(X)

# 计算加权错误率
incorrect = (ypred != y)
err = np.dot(weights, incorrect)

# 计算弱学习器权重
alpha = 0.5 * np.log((1 - err) / (err + 1e-10))

# 更新样本权重
weights = np.exp(-alpha * y * ypred)
weights /= weights.sum() # 归一化

self.alphas.append(alpha)
self.models.append(stump)

def predict(self, X):
    # 加权投票
    predictions = sum(alpha * model.predict(X)
    for alpha, model in zip(self.alphas, self.models))
    return np.sign(predictions)

```

GBDT（梯度提升决策树）：

不是修改样本权重，而是让每棵新树拟合前一轮的残差（负梯度）：

$$F_{t+1}(x) = F_t(x) + \eta \cdot h_t(x)$$

其中 h_t 是拟合残差的新树， η 是学习率（shrinkage）。

XGBoost vs LightGBM vs CatBoost：

特性 XGBoost LightGBM CatBoost

树生长策略 Level-wise（按层） Leaf-wise（按叶子） Level-wise +

分裂算法 预排序 直方图加速 直方图 + 有序提升

类别特征 需要手动编码 支持但需编码 原生支持

训练速度 较慢 快 中等

过拟合控制 较好 需注意 (Leaf-wise 易过拟合) 较好

适用场景 中小数据集 大数据集 类别特征多

LightGBM 的 Leaf-wise 策略：

Level-wise (XGBoost) : Leaf-wise (LightGBM) :

[A] [A]

/ \ / \

[B] [C] [B] [C]

/ \ / \ / \

[D] [E] [F] [G] [D] [E] [H] ← 继续分裂增益最大的叶子

[H] ← 增益更大的路径更深

Leaf-wise 优先分裂增益最大的叶子，可能长出不对称的深树，精度更高但更容易过拟合。

集成方法对比

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

随机森林 (Bagging)

```
rf = RandomForestClassifier(n_estimators=100, max_depth=10)
rf.fit(Xtrain, ytrain)
```

GBDT (Boosting)

```
gbdt = GradientBoostingClassifier(n_estimators=100)
gbdt.fit(Xtrain, ytrain)
```

XGBoost (更快的 Boosting)

```
import xgboost as xgb
dtrain = xgb.DMatrix(Xtrain, label=ytrain)
params = {'maxdepth': 6, 'eta': 0.1, 'objective': 'binary:logitmax'}
model = xgb.train(params, dtrain, num_boost_round=1000)
```

LightGBM (大数据集首选)

```
import lightgbm as lgb
```

```
traindata = lgb.Dataset(Xtrain, label=ytrain)
params = {'objective': 'binary', 'learningrate':
model = lgb.train(params, traindata, numboostroun
```

方法 原理 降低偏差 降低方差 过拟合风险

Bagging	并行训练独立模型	—	低
Boosting	序列训练互补模型	—	较高
Stacking	用元模型组合基模型		中

实战选型建议：

- 表格数据竞赛：LightGBM > XGBoost > 随机森林
- 小数据集：随机森林更稳定
- 类别特征多：CatBoost
- 需要可解释性：随机森林（特征重要性）

3.8 特征工程

特征工程是传统机器学习中最关键的一步。"数据和特征决定了机器学习的上限，模型和算法只是逼近这个上限"。

3.8.1 数值特征处理

特征缩放：不同量纲的特征会导致模型偏向数值大的特征。

方法 公式 适用场景 对异常值

StandardScaler	$(x - \mu) / \sigma$	数据近似正态分布	敏感
MinMaxScaler	$(x - x_{\min}) / (x_{\max} - x_{\min})$	数据有明确上下限	
RobustScaler	$(x - Q2) / (Q3 - Q1)$	数据有异常值	鲁棒
Log变换	$\log(1 + x)$	数据右偏 / 长尾	压缩极端值

```
from sklearn.preprocessing import StandardScaler,
import numpy as np
```

不同缩放方法的对比

```
X = np.array([[1], [2], [3], [4], [100]]) # 含异常值
```

StandardScaler: 受异常值影响, 大部分值被压缩到接近 0

```
Xstd = StandardScaler().fit_transform(X) # [-0.57,
```

MinMaxScaler: 更受异常值影响, 正常值被压缩到 0 附近

```
Xminmax = MinMaxScaler().fit_transform(X) # [0, 0.]
```

RobustScaler: 基于中位数和四分位距, 对异常值鲁棒

```
Xrobust = RobustScaler().fit_transform(X) # 推荐用于含异
```

Log 变换: 处理右偏分布 (如收入、价格)

```
Xlog = np.log1p(X) # log(1+x), 避免 log(0)
```

什么时候需要特征缩放?

算法	是否需要	原因
----	------	----

SVM、KNN、K-means	必须	依赖距离计算
-----------------	----	--------

Ridge / Lasso 回归	必须	正则化对系数大小敏感
------------------	----	------------

梯度下降优化	必须	特征尺度影响收敛速度
--------	----	------------

随机森林、决策树	不需要	基于特征分裂, 不受尺度影响
----------	-----	----------------

朴素贝叶斯	不需要	基于概率估计
-------	-----	--------

3.8.2 类别特征编码

编码方式	原理	适用场景	维度爆炸
------	----	------	------

Label Encoding	每个类别映射为整数	有序类别 (低 / 中 / 高)	无
----------------	-----------	------------------	---

One-Hot Encoding	每个类别一个二进制列	无序类别, 类别少	可能
------------------	------------	-----------	----

Target Encoding	用目标变量的均值编码	高基数类别	无
-----------------	------------	-------	---

Frequency Encoding	用类别频率编码	类别频率有意义	无
--------------------	---------	---------	---

Embedding	用低维向量表示类别	类别极多、深度学习	无
-----------	-----------	-----------	---

```
from sklearn.preprocessing import OneHotEncoder,  
import pandas as pd
```

One-Hot 编码

```
df = pd.DataFrame({'城市': ['北京', '上海', '广州', '北京']})
```

```

onehot = pd.get_dummies(df['城市'], prefix='城市') # 简单
或用 sklearn
encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df[['城市']])

Target Encoding (注意：必须在交叉验证循环内计算，否则会数据泄露！)
def target_encode_kfold(df, col, target, nsplits=5):
    """ K折 Target Encoding, 避免数据泄露 """
    from sklearn.model_selection import KFold
    df[f'{col}targetenc'] = np.nan
    kf = KFold(nsplits=nsplits, shuffle=True, random_state=None)

    for trainidx, valididx in kf.split(df):
        # 用训练集计算每个类别的目标均值
        means = df.iloc[trainidx].groupby(col)[target].mean()
        # 在验证集上编码
        df.iloc[valididx, df.columns.get_loc(f'{col}targetenc')] = df.iloc[valididx][col].map(means)

    # 全局均值填充未知类别
    globalmean = df[target].mean()
    df[f'{col}targetenc'] = df[f'{col}targetenc'].fillna(globalmean)
    return df

```

高基数特征示例：邮政编码、用户ID

One-Hot → 维度爆炸（几万列）

Target Encoding → 压缩为1列，信息保留好

3.8.3 特征选择

为什么要特征选择？

- 降维：减少计算开销
- 去噪：去除无关特征，提高泛化
- 可解释性：找到最重要的因素

方法 原理 有监督 计算成本 稳定性

方差阈值 删除方差很小的特征 低 高

相关系数 删除与目标相关性低的特征 低 中

SelectKBest 选择统计检验最显著的K个特征 低 中

递归特征消除 (RFE) 反复训练模型, 去掉最不重要的特征 高 低

L1正则化 通过Lasso自动选择特征 中 中

树模型特征重要性 基于树模型的分裂增益 中 高

```
from sklearn.featureselection import SelectKBest,  
from sklearn.ensemble import RandomForestClassifier
```

方法1: 统计检验选择

```
selector = SelectKBest(fclf, k=20)  
Xselected = selector.fit_transform(X, y)  
selectedfeatures = selector.getsupport(indices=Tr
```

方法2: 基于模型的特征重要性

```
rf = RandomForestClassifier(n_estimators=100)  
rf.fit(X, y)  
importances = pd.Series(rf.featureimportances, in  
topfeatures = importances.nlargest(20).index.tolist
```

方法3: 递归特征消除 (RFE)

```
rfe = RFE(estimator=RandomForestClassifier(nestin  
Xselected = rfe.fit_transform(X, y)
```

方法4: L1正则化特征选择

```
from sklearn.linearmodel import Lasso  
lasso = Lasso(alpha=0.01)  
lasso.fit(X, y)  
selected = np.where(lasso.coef != 0)[0] # 非零系数对应的
```

3.8.4 缺失值处理

方法 适用场景 风险

删除 缺失比例 > 50% 丢失信息

均值 / 中位数填充 数值特征, 缺失少 降低方差

众数填充 类别特征 可能引入偏差

插值法 时间序列 假设连续性

模型预测填充 缺失与其它特征相关 计算成本高

标记缺失 缺失本身有含义 增加维度

缺失值处理最佳实践

```
from sklearn.impute import SimpleImputer, KNNImputer
```

1. 简单填充 (快速基线)

```
imputer = SimpleImputer(strategy='median') # 数值特征
X_imputed = imputer.fit_transform(X)
```

2. KNN填充 (更精确, 利用相似样本)

```
knn_imputer = KNNImputer(n_neighbors=5)
X_imputed = knn_imputer.fit_transform(X)
```

3. 标记缺失 (缺失本身可能是信号)

```
df['featurewasmissing'] = df['feature'].isna().astype(int)
df['feature'] = df['feature'].fillna(df['feature'].mode()[0])
```

3.8.5 完整 Pipeline

```
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
```

区分数值特征和类别特征

```
numeric_features = ['age', 'income', 'score']
categorical_features = ['city', 'gender', 'education']
```

数值特征处理管道

```
numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())])
```

```
( ' s c a l e r ' ,   S t a n d a r d S c a l e r ( ) )  
] )
```

类别特征处理管道

```
c a t e g o r i c a l t r a n s f o r m e r   =   P i p e l i n e ( [  
( ' i m p u t e r ' ,   S i m p l e I m p u t e r ( s t r a t e g y = ' m o s t f r e q u e n t ' ,  
( ' e n c o d e r ' ,   O n e H o t E n c o d e r ( h a n d l e u n k n o w n = ' i g n o r e ' ,  
] )
```

合并预处理

```
p r e p r o c e s s o r   =   C o l u m n T r a n s f o r m e r ( [  
( ' n u m ' ,   n u m e r i c t r a n s f o r m e r ,   n u m e r i c f e a t u r e s ) ,  
( ' c a t ' ,   c a t e g o r i c a l t r a n s f o r m e r ,   c a t e g o r i c a l f e a t u r e s )  
] )
```

完整 Pipeline (预处理 + 模型)

```
p i p e l i n e   =   P i p e l i n e ( [  
( ' p r e p r o c e s s o r ' ,   p r e p r o c e s s o r ) ,  
( ' c l f ' ,   R a n d o m F o r e s t C l a s s i f i e r ( n e s t i m a t o r s = 1 0 0 ) )  
] )
```

使用

```
p i p e l i n e . f i t ( X t r a i n ,   y t r a i n )  
y p r e d   =   p i p e l i n e . p r e d i c t ( X t e s t )
```

Pipeline的优势：

- 1 . 防止数据泄露（预处理在C V内部执行）
- 2 . 代码简洁，易于部署
- 3 . 可序列化保存

- - -

四、无监督学习详解

4 . 1 K - m e a n s 详解

算法步骤：

- 1 . 随机初始化K个聚类中心
- 2 . 将每个样本分配到最近的聚类中心
- 3 . 更新聚类中心为所属样本的均值
- 4 . 重复 2 - 3 直到收敛

数学目标：最小化簇内距离之和

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

算法的收敛性：K - m e a n s 保证每次迭代J不增加（因为步骤2和步骤3都使J减小或不变），但只能收敛到局部最优，不保证全局最优。

K - m e a n s ++ 初始化：改进的初始化策略，选择距离已有中心尽量远的新中心。

K - m e a n s ++ 初始化过程：

- 1 . 随机选一个样本作为第一个中心 μ_1
- 2 . 对每个样本x，计算到最近中心的距离 $D(x) = \min_i \|x - \mu_i\|$
- 3 . 以概率 $D(x)^2 / \sum D(x')^2$ 选择下一个中心（距离越远越可能被选）
- 4 . 重复 2 - 3 直到选出K个中心

```
from sklearn.cluster import KMeans
import numpy as np
```

K - m e a n s

```
kmeans = KMeans(nclusters=3, init='k-means++', n_init=10)
labels = kmeans.fit_predict(X)
```

关键属性

```
print(f"聚类中心: {kmeans.cluster_centers}")
print(f"簇内距离之和 (Inertia): {kmeans.inertia:.2f}")
print(f"迭代次数: {kmeans.n_iter}")
```

选择最优K值

确定K值的方法：

方法	原理	优点	缺点
----	----	----	----

肘部法则 画 `Inertia vs K` 曲线, 选拐点 直观 拐点不明显

轮廓系数 衡量聚类的紧密度和分离度 有客观标准 计算成本高

`Gap Statistic` 比较实际与随机分布的 `Inertia` 理论更严谨 需要生成参考分布
业务需求 根据业务场景决定 `K` 最实用 主观

肘部法则

```
inertias = []
Krange = range(1, 11)
for k in Krange:
    km = KMeans(nclusters=k, ninit=10)
    km.fit(X)
    inertias.append(km.inertia)
```

画图: 选择拐点处的 `K` 值

```
import matplotlib.pyplot as plt
plt.plot(Krange, inertias, 'bo-')
plt.xlabel('K')
plt.ylabel('Inertia')
```

轮廓系数 (-1到1, 越大越好)

```
from sklearn.metrics import silhouettescore
for k in range(2, 11):
    km = KMeans(nclusters=k, ninit=10)
    labels = km.fit_predict(X)
    score = silhouettescore(X, labels)
    print(f"K = {k}, 轮廓系数 = {score:.4f}")
```

`K-means` 的局限与替代方案:

局限 原因 替代方案

只能发现球形簇 使用欧氏距离 `DBSCAN` (任意形状)

需要预设 `K` 算法本身不知道 `K` `DBSCAN`、层次聚类

对异常值敏感 均值受异常值影响 `K-medoids` (用中位数点)

各簇大小不同时效果差 等概率假设 `GMM` (高斯混合模型)

DBSCAN详解：

DBSCAN核心概念：

- 核心点：半径 ϵ 内至少有 `min_samples` 个邻居
- 边界点：在核心点的 ϵ 半径内，但自身邻居不足
- 噪声点：既非核心也非边界 → 被视为异常值

优势：自动确定簇数、能发现任意形状、识别异常值

```
from sklearn.cluster import DBSCAN
```

DBSCAN聚类

```
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X)
label = -1 表示噪声点（异常值）
```

调参：`eps` 太小→太多噪声点；`eps` 太大→所有点变成一个簇

可用 `k-distance` 图选择 `eps`

```
from sklearn.neighbors import NearestNeighbors
neighbors = NearestNeighbors(nneighbors=5)
neighbors.fit(X)
distances, = neighbors.kneighbors(X)
kdistances = np.sort(distances[:, -1])
```

画图，选拐点处作为 `eps`

GMM（高斯混合模型）：

K-means 的 "软分类" 版本，每个样本以概率属于每个簇：

$$P(x_i \in C_k) = \frac{p_{ik}}{\sum_j p_{ij}}$$

```
from sklearn.mixture import GaussianMixture
```

GMM聚类（软分类）

```
gmm = GaussianMixture(ncomponents=3, covariance_type='full')
labels = gmm.fit_predict(X)
```

```
probs = gmm.predict_proba(X) # 每个样本属于各簇的概率
```

选择最佳K值（用BIC / AIC，越小越好）

```
bics = []
for k in range(1, 11):
    gmm = GaussianMixture(ncomponents=k)
    gmm.fit(X)
    bics.append(gmm.bic(X))
bestk = np.argmin(bics) + 1
```

4.2 PCA 详解

PCA的两种等价理解：

1. 找到使数据方差最大的正交方向（最大化投影方差）
2. 找到使重构误差最小的低维表示（最小化重构误差）

PCA的数学推导：

1. 数据中心化： $\tilde{X} = X - \mu$
2. 计算协方差矩阵： $C = \frac{1}{n} \tilde{X}^T \tilde{X}$
3. 特征分解： $C = V \Lambda V^T$
4. 选择前k个最大特征值对应的特征向量 V_k
5. 投影： $Z = \tilde{X} V_k$

为什么最大特征值对应的方向方差最大？

协方差矩阵 C 的第i个特征值 λ_i

恰好等于数据在第i个特征向量方向上的方差。因此选择最大特征值对应的特征向量，就是选择方差最大的方向

特征值与方差解释比例：

$\text{第 } i \text{ 主成分的方差解释比例} = \frac{\lambda_i}{\sum_{j=1}^k \lambda_j}$

例如：3个特征值 $[4, 1, 0.5]$ ，则第一主成分解释了 $4 / (4 + 1 + 0.5) = 72\%$

PCA的步骤详解与直觉：

原始数据（2D）： PCA投影后（1D）：

```

. . .
. . . . → . . . .
. . . . . . . .
. . . .

```

找到方差最大的方向（红线），将数据投影到该方向上
投影后的数据虽然只有1维，但保留了最多的信息

```

from sklearn.decomposition import PCA
import numpy as np

```

PCA降维

```

pca = PCA(ncomponents=0.95) # 保留95%的方差（自动选择维度）
Xreduced = pca.fittransform(X)
print(f"原始维度: {X.shape[1]}")
print(f"降维后: {Xreduced.shape[1]}")
print(f"解释方差比: {pca.explainedvarianceratio.sum():.4f}")

```

查看每个主成分的解释方差

```

for i, ratio in enumerate(pca.explainedvarianceratio):
    print(f"PC{i+1}: {ratio:.4f} ({ratio/pca.explainedvarianceratio.sum():.4f})")

```

也可以指定维度

```

pca2d = PCA(ncomponents=2)
X2d = pca2d.fittransform(X) # 用于2D可视化

```

PCA的逆变换（重建）

```

Xreconstructed = pca.inversetransform(Xreduced)
reconstructionerror = np.mean((X - Xreconstructed)**2)
print(f"重建MSE: {reconstructionerror:.6f}")

```

PCA完整实现（从零开始）：

```

class MyPCA:
    """PCA的完整实现"""
    def __init__(self, ncomponents=None):

```

```

self.ncomponents = ncomponents

def fit(self, X):
    # 1. 中心化
    self.mean = X.mean(axis=0)
    Xcentered = X - self.mean

    # 2. 计算协方差矩阵
    n = X.shape[0]
    cov = Xcentered.T @ Xcentered / (n - 1)

    # 3. 特征分解
    eigenvalues, eigenvectors = np.linalg.eigh(cov)

    # 4. 按特征值从大到小排序
    sortedidx = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[sortedidx]
    eigenvectors = eigenvectors[:, sortedidx]

    # 5. 选择前k个
    if self.ncomponents is None:
        self.ncomponents = X.shape[1]
    self.components = eigenvectors[:, :self.ncomponents]
    self.explainedvariance = eigenvalues[:self.ncomponents]
    self.explainedvarianceratio = self.explainedvariance / eigenvalues.sum()

    return self

def transform(self, X):
    return (X - self.mean) @ self.components.T

def fittransform(self, X):
    self.fit(X)
    return self.transform(X)

def inversetransform(self, Z):
    return Z @ self.components + self.mean

```

P C A 的应用场景：

场景 目的 说明

数据可视化 降到 2 - 3 维 高维数据无法直接可视化

降噪 去掉小特征值方向 噪声通常分布在方差小的方向

特征压缩 减少特征数 加速模型训练，减少内存

多重共线性 去除特征相关性 P C A 后各主成分正交（不相关）

人脸识别 E i g e n f a c e s 人脸图像降维为 " 特征脸 "

P C A v s 其他降维方法：

方法 类型 保持全局结构 保持局部结构 速度 可逆

P C A 线性 快

t - S N E 非线性 慢

U M A P 非线性 部分 中

A u t o e n c o d e r 非线性 部分 部分 中

4 . 3 t - S N E 详解

核心思想：使高维空间中相似的点在低维空间中也相似，不相似的点也不相似。

算法原理：

1 . 高维空间：用高斯分布计算点对之间的相似度（条件概率）

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_k \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

2 . 低维空间：用 t 分布（长尾）计算点对之间的相似度

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_k (1 + \|y_i - y_k\|^2)^{-1}}$$

3 . 优化目标：最小化两个分布的 K L 散度

$$C = \sum_i KL(p_i || q_i) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

为什么用 t 分布而不是高斯分布？

高维空间的距离在低维空间会 " 拥挤 " ——很多点对会被映射到相似的距离。 t 分布的长尾特性使得中等距离的

点在低维空间被推得更远，从而更好地展示簇的分离。

关键参数：

- `perplexity`：控制有效邻居数量（5 - 50，通常30），可理解为每个点期望有多少个“近邻”
- `learning rate`：通常10 - 1000，太大则损失函数震荡，太小则陷入局部最优
- `niter`：至少1000次迭代

重要注意事项：

1. `t-SNE` 结果不可复现（随机初始化）
2. 簇间距离不可比较
3. 不能用于新数据的投影
4. 主要用于可视化，不是特征提取
5. 建议先用 PCA 降到 50 维再用 `t-SNE`（加速 + 降噪）

```
from sklearn.manifold import TSNE
```

先 PCA 降噪，再 `t-SNE` 可视化

```
pca = PCA(ncomponents=50) # 先降到 50 维
```

```
Xpca = pca.fit_transform(X)
```

```
tsne = TSNE(ncomponents=2, perplexity=30, learning_rate=150, random_state=1234)
```

```
X2d = tsne.fit_transform(Xpca)
```

可视化

```
plt.scatter(X2d[:, 0], X2d[:, 1], c=labels, cmap=plt.cm.nipy_spectral)
```

4.4 UMAP 简介

UMAP 是 `t-SNE` 的更快替代，且更好地保持全局结构：

```
import umap
```

UMAP 降维

```
reducer = umap.UMAP(nneighbors=15, mindist=0.1, n_epochs=1000)
```

```
X2d = reducer.fit_transform(X)
```

参数：

`n n e i g h b o r s` : 类似 `p e r p l e x i t y` , 控制局部 v s 全局结构的平衡

小值 (5) → 更关注局部结构 ; 大值 (50) → 更关注全局结构

`m i n d i s t` : 低维空间中点的最小距离

小值 (0 . 001) → 簇更紧密 ; 大值 (0 . 5) → 簇更分散

`t - S N E` v s `U M A P` 选择 :

- 数据量 < 10K → 两者都可以
- 数据量 > 100K → `U M A P` (`t - S N E` 太慢)
- 需要保持全局结构 → `U M A P`
- 需要可视化效果更好 → 试两者, 选效果好的

- - -

五、模型评估详解

5 . 1 混淆矩阵

预测正 预测负

实际正 `T P` `F N`

实际负 `F P` `T N`

指标 公式 直觉 关注场景

准确率 (`A c c u r a c y`) $(T P + T N) / (T P + F P + F N + T N)$ 整体正确率 类别均衡时

精确率 (`P r e c i s i o n`) $T P / (T P + F P)$ 预测为正中真正为真的比例 垃圾邮件 (不能误杀)

召回率 (`R e c a l l`) $T P / (T P + F N)$ 真正为正中被正确预测的比例 癌症检测 (不能漏诊)

`F 1` 分数 $2 P R / (P + R)$ `P` 和 `R` 的调和平均 `P` 和 `R` 都重要时

`F β` 分数 $(1 + β^2) P R / (β^2 P + R)$ $β > 1$ 更重 `R`, $β < 1$ 更重 `P` 可调权重

准确率的陷阱 : 当数据极度不平衡时, 准确率会误导。

例如 : 1000 个样本中 990 个负样本, 10 个正样本。全预测为负, 准确率仍有 99% ! 此时应关注 `F 1`、

不同场景的关注点 :

场景 更关注 原因

垃圾邮件 精确率 不能把正常邮件误判为垃圾邮件

癌症检测 召回率 不能漏掉任何一例癌症

搜索排序 精确率@K 前10个结果要准确

欺诈检测 召回率 宁可误报不可漏报

5.2 ROC曲线与AUC

ROC曲线：以FPR为横轴、TPR为纵轴画的曲线。通过调整分类阈值，得到一系列(FPR, TPR)

$TPR(\text{真阳性率}) = TP / (TP + FN) = \text{召回率}$

$FPR(\text{假阳性率}) = FP / (FP + TN) = \text{误报率}$

理想情况： $TPR = 1$, $FPR = 0 \rightarrow$ 左上角

随机猜测：沿对角线

- $AUC = 1.0$ ：完美分类器
- $AUC = 0.5$ ：随机猜测
- $AUC > 0.9$ ：优秀分类器
- $0.7 < AUC < 0.9$ ：中等分类器

AUC的概率解释：AUC等于随机取一个正样本和一个负样本，分类器将正样本排在负样本前面的概率。

ROC vs PR曲线：

维度 ROC曲线 PR曲线

适用 类别相对均衡 类别严重不平衡

关注 整体分类能力 正类表现

对不平衡敏感 不敏感 敏感

基准线 对角线 ($AUC = 0.5$) 水平线 ($P = \text{正类比例}$)

```
from sklearn.metrics import (roccurve, auc, class
precisionrecallcurve, averageprecisionscore)
```

完整的评估流程

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
```

分类报告

```
print(classificationreport(ytest, ypred))
```

ROC曲线

```
fpr, tpr, thresholdsroc = roccurve(ytest, yprob)
rocauc = auc(fpr, tpr)
print(f"AUC: {rocauc:.4f}")
```

PR曲线（类别不平衡时更推荐）

```
precision, recall, thresholdspr = precisionrecall(ytest, yprob)
ap = averageprecisionscore(ytest, yprob)
print(f"AP: {ap:.4f}")
```

找到最优阈值（Youden's J = TPR - FPR 的最大值）

```
j_scores = tpr - fpr
bestthreshold = thresholdsroc[np.argmax(j_scores)]
print(f"最优阈值: {bestthreshold:.4f}")
```

5.3 交叉验证

为什么需要交叉验证？

单次划分训练 / 测试集的结果依赖于划分方式，可能运气好或差。交叉验证通过多次划分获得更可靠的评估。

验证方式 原理 适用场景 计算量

Hold-out 单次划分 数据量大 1 x

K-Fold 分K份，轮流作测试集 通用 K x

Stratified K-Fold 保持各类比例的K-Fold 分类问题（尤其不平衡） K x

Time Series Split 按时间划分，不泄露未来信息 时间序列 K x

Leave-One-Out 每次留1个样本 数据极少 N x

```
from sklearn.modelselection import crossvalscore,
TimeSeriesSplit
```

K折交叉验证

```
scores = crossvalscore(model, X, y, cv=5, scoring='f1')
print(f"5折交叉验证: {scores.mean():.3f} ± {scores.std():.3f}")
```

分层K折（保持各类别比例）

```
s k f  =  S t r a t i f i e d K F o l d ( n s p l i t s = 5 ,   s h u f f l e = T r u e ,   r a
```

时间序列交叉验证

```
t s c v  =  T i m e S e r i e s S p l i t ( n s p l i t s = 5 )
```

每次用前面的数据训练，后面的数据测试，不泄露未来信息

网格搜索 + 交叉验证

```
p a r a m g r i d  =  {  
' n e s t i m a t o r s ' :   [ 5 0 ,   1 0 0 ,   2 0 0 ] ,  
' m a x d e p t h ' :   [ 5 ,   1 0 ,   2 0 ] ,  
' l e a r n i n g r a t e ' :   [ 0 . 0 1 ,   0 . 1 ,   0 . 2 ]  
}  
  
g r i d s e a r c h  =  G r i d S e a r c h C V (   
    G r a d i e n t B o o s t i n g C l a s s i f i e r ( ) ,  
    p a r a m g r i d ,   c v = 5 ,   s c o r i n g = ' f 1 ' ,   n j o b s = - 1  
)  
  
g r i d s e a r c h . f i t ( X t r a i n ,   y t r a i n )  
p r i n t ( f " 最佳参数： { g r i d s e a r c h . b e s t p a r a m s } " )  
p r i n t ( f " 最佳分数： { g r i d s e a r c h . b e s t s c o r e : . 4 f } " )
```

数据泄露的常见场景：

场景 泄露方式 正确做法

特征缩放 在整个数据集上 `fit` 在CV循环内 `fit`

特征选择 在整个数据集上选特征 在CV循环内选

`Target Encoding` 用全部数据计算目标均值 用训练集计算

过采样 在整个数据集上 `SMOTE` 只在训练集上 `SMOTE`

正确的做法：所有预处理都在 `Pipeline` 内

```
f r o m   s k l e a r n . p i p e l i n e   i m p o r t   P i p e l i n e  
f r o m   s k l e a r n . p r e p r o c e s s i n g   i m p o r t   S t a n d a r d S c a l e r  
f r o m   s k l e a r n . m o d e l s e l e c t i o n   i m p o r t   c r o s s v a l s c o r e  
  
p i p e l i n e  =  P i p e l i n e ( [  
    ( ' s c a l e r ' ,   S t a n d a r d S c a l e r ( ) ) ,   # 在CV内部 fit
```

```
( ' c l f ' ,   L o g i s t i c R e g r e s s i o n ( ) )  
] )
```

这样 `crossvalscore` 会自动在每折内独立地 `fit scaler`
`scores = crossvalscore(pipeline, X, y, cv=5)`
- - -

六、新兴交叉范式

范式 说明 典型方法

半监督学习 少量标签 + 大量无标签 `FixMatch`（一致性正则 + 伪标签）、`MixMatch`

自监督学习 从数据自身构造监督信号 `BERT`（掩码预测）、`SimCLR`（对比学习）、`GPT`（下一词

主动学习 模型主动选择最有价值的样本标注 不确定性采样、多样性采样

迁移学习 将一个领域学到的知识迁移到新领域 预训练 + 微调（当前最成功的方式）

联邦学习 多方数据不共享的联合训练 `FedAvg`

现代 AI 系统的多范式融合：

- 自动驾驶：监督学习（感知）+ 无监督学习（地图构建）+ 强化学习（决策规划）
- `ChatGPT`：自监督预训练 + 监督微调 + 强化学习（`RLHF`）

- - -

七、如何选择合适的算法？

7.1 决策流程

1. 数据量多大？

- ├—— 很小（< 1000） → 传统 ML（SVM、随机森林）
- └—— 很大（> 100K） → 深度学习或 `XGBoost`

2. 有标签吗？

- ├—— 有 → 监督学习
- ├—— 无 → 无监督学习
- └—— 少量 → 半监督 / 自监督学习

3. 任务类型？

- |—— 分类 → 逻辑回归 / 随机森林 / 神经网络
- |—— 回归 → 线性回归 / X G B o o s t / 神经网络
- |—— 聚类 → K - m e a n s / D B S C A N

4 . 可解释性要求？

- |—— 高 → 决策树 / 线性模型
- |—— 低 → 随机森林 / 深度学习

5 . 实时性要求？

- |—— 高 → 线性模型 / 小神经网络
- |—— 低 → 复杂模型 / 集成方法

7 . 2 常见问题排查

问题 可能原因 排查方法

训练集表现差 模型太简单、特征不足 增加模型复杂度、添加特征

测试集表现差 过拟合 正则化、增加数据、早停

训练不收敛 学习率不当、梯度问题 调小学习率、检查数据

训练太慢 模型太大、数据太多 减小模型、增大 b a t c h、用 G P U

- - -

核心总结 * * ：监督学习依赖 " 上帝视角 " 的标注；无监督学习只靠数据的内在结构；强化学习只能靠 " 试错反 " 摸索。理解三大范式的根本差异——数据流、反馈信号、目标函数——是选择正确方法的关键。偏差 - 方差树贯穿所有机器学习问题的核心线索。